

PES UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
LAB ASSIGNMENT: DATA VERSION CONTROL (DVC) AND MLFLOW

Student Name:	Aarav Yuval B G
SRN:	PES1UG24AM004
Faculty / Evaluator:	Sunaina Kumar
Course / Semester:	Software Engineering Lab (5th Semester)

1. Executive Summary & Lab Objectives

Machine Learning operations (MLOps) necessitate versioning both source code and heavy data assets. Git fails when handling multi-megabyte/gigabyte binary datasets due to repository bloat and file limits. This laboratory assignment demonstrates the complementary use of **Data Version Control (DVC)** and **MLflow**. DVC provides content-addressable storage and lightweight Git pointers, while MLflow tracks experiments, hyperparameters, performance metrics, and packaged models. Linking the DVC dataset checksum into MLflow achieves 100% reproducible pipelines.

2. Part 1: Data Version Control (DVC) Implementation

DVC was initialized with `dvc init`. A data versioning lifecycle was simulated using `data.txt`, tracking version updates, Git commits, version tags (v1.0 and v2.0), and double-checkout rollbacks.

Stage	Commands Executed	Outcome / Checksum
Init	<code>git init</code> <code>dvc init</code> <code>git commit -m 'Init DVC'</code>	Initialized <code>.dvc/</code> , created <code>.dvcignore</code> .
Version 1.0	<code>echo 'version 1' > data.txt</code> <code>dvc add data.txt</code> <code>git commit -m 'v1' & tag v1.0</code>	MD5: 70714828ec652824f0bd76e1e2da849e Saved to <code>.dvc/cache</code> .
Version 2.0	<code>echo 'version 2' > data.txt</code> <code>dvc add data.txt</code> <code>git commit -m 'v2' & tag v2.0</code>	MD5: 75dafceb0d26a67df209575587f1334a Pointer file updated.
Double Checkout	<code>git checkout v1.0</code> <code>dvc checkout</code>	Workspace synced to 'version 1' data instantaneously from cache.
Remote Storage	<code>dvc remote add -d localremote <path></code> <code>dvc push</code>	Cache pushed to external storage directory.

3. Part 2: MLflow Experiment Tracking & MLOps Bridge

Four modules were developed and executed to showcase the full MLflow capability suite:

- **02_mlflow_dvc_pipeline.py**: Retrieves DVC hash (21d441a28bce4417276097df955afc50) and logs it into MLflow. Runs baseline vs tuned models.
- **03_mlflow_autolog.py**: Demonstrates zero-effort tracking via `mlflow.sklearn.autolog()`.
- **04_mlflow_wine.py**: Explicit logging on 13-feature Wine dataset achieving 100% test accuracy.
- **05_test_inference.py**: Restores serialized model artifact from MLflow storage and executes live inference predictions.

4. Key Results & Experiment Logs

Experiment	Run Name	Logged Params (inc. DVC Hash)	Accuracy
DVC_MLflow_Pipeline	Run_1_Baseline	<code>n_estimators=30, max_depth=2</code> DVC: 21d441a28bce...	92.11%
DVC_MLflow_Pipeline	Run_2_Tuned	<code>n_estimators=120, max_depth=5</code> DVC: 21d441a28bce...	92.11%

MLflow_Quickstart	Autolog_LogisticRegression	solver='lbfgs', max_iter=1000	100.0%
Wine_Classification	Wine_RandomForest_Run	n_estimators=100, max_depth=3	100.0%

5. Summary & Viva Insights

Key Takeaway: DVC handles data versioning without Git bloat using MD5 checksums. MLflow handles experiment metrics and models. Together, logging the DVC hash into MLflow guarantees total reproducibility. The double checkout (git checkout <tag> && dvc checkout) seamlessly restores corresponding data states.